

Object-Oriented Modeling of Electronic Circuits, Spring 1405

Homework 3

SystemC RTL and BFM – Covering materials through Class 8

Ali Dehghani

810103414

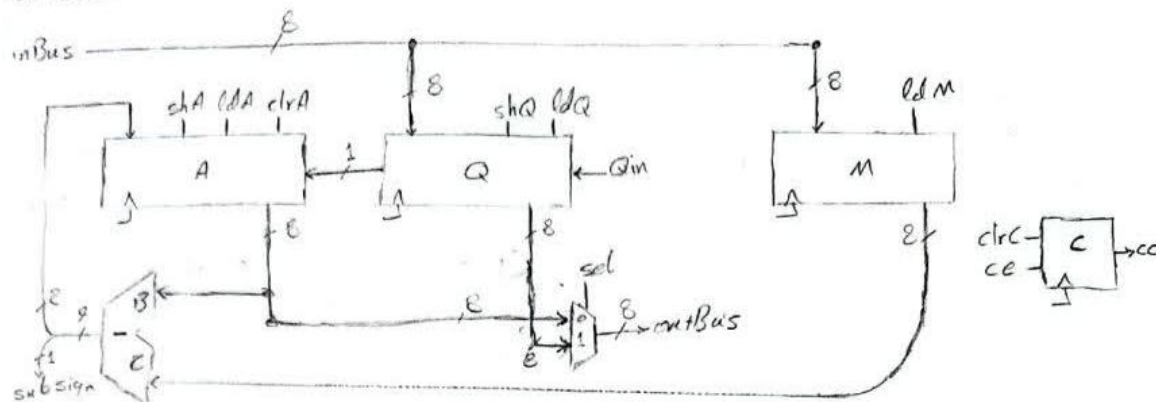
Armin Nezafat

810103558

Q1.

a. Show the RTL schematic of the datapath for the divider.

Part 1: a.



A: 8bit shift register with clear

Q: 8bit shift register

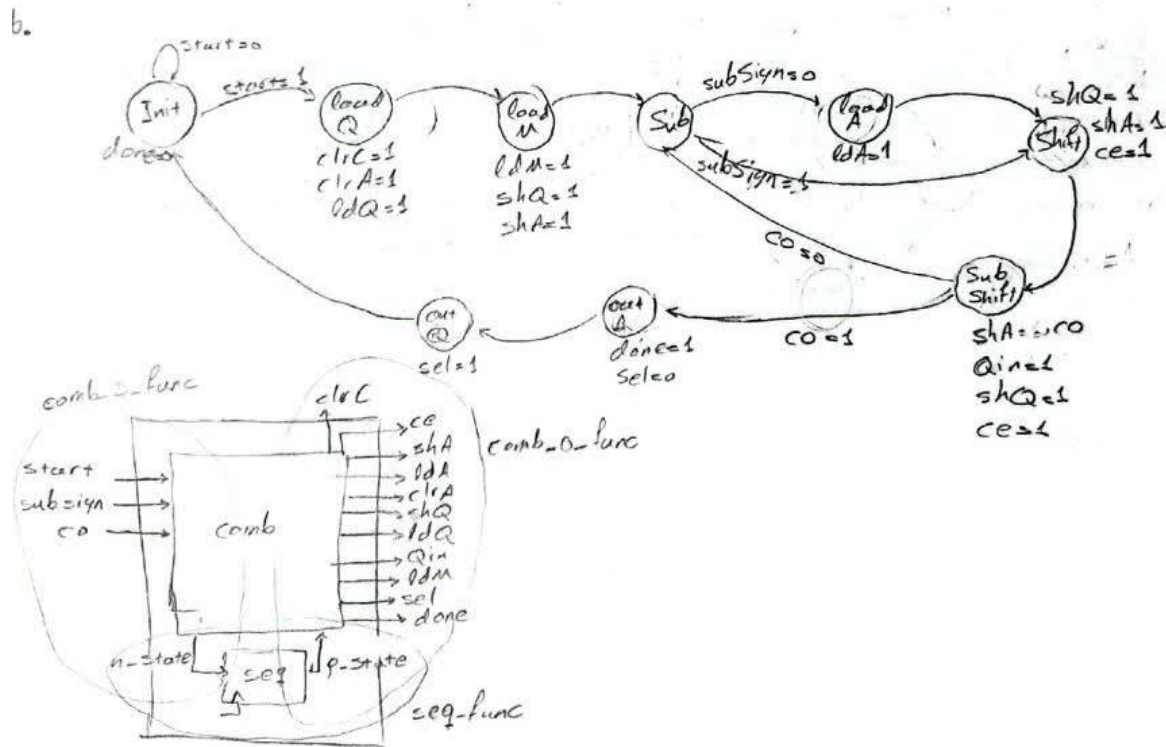
M: 8bit register

3bit Subtractor: $out = B - C$ ($B = \{A[6:0], Q[7]\}$)

C: 3bit up-counter with clear and count enable (for count n)

نویسندگان: علی دهگانی، آرمن نضاف
استاد راهنما: دکتر سید علی حسینی

b. Show the controller for the divider.



c. Write system-c description for the datapath.

```

#include <systemc.h>
#include "shiftRegisterClr.h"
#include "shiftRegister.h"
#include "register.h"
#include "subtractor.h"
#include "upCounter.h"
#include "mux.h"

SC_MODULE(datapath)
{
    // Port Declaration
    sc_in<sc_logic> clk;
    sc_in<sc_logic> ldA, shA, clrA;
    sc_in<sc_logic> ldQ, shQ;
    sc_in<sc_logic> ldM;
    sc_in<sc_logic> Qin;
    sc_in<sc_logic> clrC, countEnable;
    sc_in<sc_logic> muxSelect;
    sc_in<sc_lv<8>> inBus;

```

```

sc_out<sc_logic> co;
sc_out<sc_logic> subSign;
sc_out<sc_lv<8> > outBus;

// Signal Declaration
sc_signal<sc_logic> q_msb;
sc_signal<sc_lv<9> > subResult;
sc_signal<sc_lv<8> > inA;
sc_signal<sc_lv<8> > outA;
sc_signal<sc_lv<8> > outQ;
sc_signal<sc_lv<8> > outM;
sc_signal<sc_lv<8> > subIn1;
sc_out<sc_lv<3>> countResult_out;
sc_signal<sc_logic> Qsout;
sc_signal<sc_logic> Asin;
sc_signal<sc_logic> Asout;

// Instantiation
shiftRegisterClr_8bit *A;
shiftRegister_8bit *Q;
register_8bit *M;
subtractor_8bit *sub;
upCounter_3bit *C;
mux_8bit *mux;

SC_CTOR(datapath)
{
    A = new shiftRegisterClr_8bit("A_shiftRegister");
        A->inData(inA);
        A->sh(shA);
        A->ld(ldA);
        A->clr(clrA);
        A->clk(clk);
        A->sin(q_msb);
        A->sout(Asout);
        A->outDate(outA);

    Q = new shiftRegister_8bit("Q_shiftRegister");
        Q->inData(inBus);
        Q->clk(clk);
        Q->ld(ldQ);
        Q->sh(shQ);
        Q->sin(Qsin);
        Q->sout(Qsout);
        Q->outData(outQ);

```

```

    M = new register_8bit("M_register");
    M->inData(inBus);
    M->ld(ldM);
    M->clk(clk);
    M->outData(outM);

    sub = new subtractor_8bit("subtractor");
    sub->AData(outA);
    sub->BData(outM);
    sub->outData(subResult);

    C = new upCounter_3bit("C_upCounter");
    C->clk(clk);
    C->clr(clrC);
    C->ce(countEnable);
    C->co(co);
    C->outData(countResult_out);

    mux = new mux_8bit("mux");
    mux->inA(outA);
    mux->inB(outQ);
    mux->sel(muxSelect);
    mux->outData(outBus);

    SC_METHOD(subResultAssign);
    sensitive << subResult;

    SC_METHOD(extract_q_msb);
    sensitive << outQ;
    SC_METHOD(checkco);
    sensitive << co << countResult_out << countEnable << subResult;
}

void subResultAssign()
void extract_q_msb();
};

```

```

#include "dividerDatapath.h"

void datapath::subResultAssign()
{
    sc_lv<9> result = subResult.read();
    subSign.write(result.get_bit(8));
}

```

```

}

void datapath::extract_q_msb()
{
    q_msb.write(outQ.read()[7]);
}

```

d. Write system-c description for the Huffman controller.

```

#ifndef DIVIDERCONTROLLER_H
#define DIVIDERCONTROLLER_H

#include <systemc.h>

SC_MODULE(controller)
{
    sc_in <sc_logic> rst, clk, start, co, subSign;
    sc_out <sc_logic> clrA, clrC, ldQ;
    sc_out <sc_logic> ldM;
    sc_out <sc_logic> shA, Qin, ldA, shQ, ce;
    sc_out <sc_logic> sel;
    sc_out <sc_logic> done;
    sc_in<sc_lv<3>> countResult;

    enum states {INIT, SUB_SHIFT, LOAD_Q, LOAD_M, LOAD_A, SHIFT_Q, SHIFT, SUB,
WAIT3, WAIT2, OUT_Q, OUT_A};

    sc_signal <states> p_state, n_state;

    SC_CTOR(controller)
    {
        SC_METHOD(comb_S_func);
        sensitive << start << co << p_state<<subSign;

        SC_METHOD(comb_O_func);
        sensitive << p_state << subSign;

        SC_METHOD(seq_func);
        sensitive << clk.pos() << rst;
    }

    void comb_S_func();
    void comb_O_func();

```

```
    void seq_func();  
};  
  
#endif
```

```
#include "dividerController.h"  
  
void controller::comb_0_func() {  
    clrA.write(sc_logic_0);  
    clrC.write(sc_logic_0);  
    ldQ.write(sc_logic_0);  
    ldM.write(sc_logic_0);  
    shA.write(sc_logic_0);  
    Qin.write(sc_logic_0);  
    ldA.write(sc_logic_0);  
    shQ.write(sc_logic_0);  
    ce.write(sc_logic_0);  
    sel.write(sc_logic_0);  
    done.write(sc_logic_0);  
  
    switch (p_state.read()) {  
        case INIT:  
            done.write(sc_logic_0);  
            break;  
        case LOAD_Q:  
            clrA.write(sc_logic_1);  
            clrC.write(sc_logic_1);  
            ldQ.write(sc_logic_1);  
            break;  
        case LOAD_M:  
            ldM.write(sc_logic_1);  
            shQ.write(sc_logic_1);  
            shA.write(sc_logic_1);  
            break;  
        case SHIFT:  
            shQ.write(sc_logic_1);  
            shA.write(sc_logic_1);  
            ce.write(sc_logic_1);  
            break;  
        case LOAD_A:  
            ldA.write(sc_logic_1);  
            break;  
        case SUB_SHIFT:
```

```

        if ((co.read()) != sc_logic_1) {
            shA.write(sc_logic_1);
        }
        else
            Qin.write(sc_logic_1);
        shQ.write(sc_logic_1);
        ce.write(sc_logic_1);
        break;
    case SUB:
        break;
    case OUT_A:
        done.write(sc_logic_1);
        sel.write(sc_logic_0);
        break;
    case OUT_Q:
        done.write(sc_logic_1);
        sel.write(sc_logic_1);
        break;
    }
}

void controller::comb_S_func() {
    n_state.write(INIT);
    switch (p_state.read()) {
        case INIT:
            if (start.read() == sc_logic_1) {
                n_state.write(LOAD_Q);
            } else {
                n_state.write(INIT);
            }
            break;
        case LOAD_Q:
            n_state.write(LOAD_M);
            break;
        case LOAD_M:
            n_state.write(SUB);
            break;
        case LOAD_A:
            n_state.write(SUB_SHIFT);
            break;
        case SHIFT:
            if ((co.read()) == sc_logic_1)
                n_state.write(OUT_A);
            n_state.write(SUB);
            break;
    }
}

```

```

        case SUB_SHIFT:
            if ((co.read()) == sc_logic_1)
                n_state.write(OUT_A);
            else
                n_state.write(SUB);
            break;
        case SUB:
            if (subSign.read() == sc_logic_0) {
                n_state.write(LOAD_A);
            } else {
                n_state.write(SHIFT);
            }
            break;
        case OUT_A:
            n_state.write(OUT_Q);
            break;
        case OUT_Q:
            n_state.write(INIT);
            break;
    }
}

void controller::seq_func() {
    if (rst.read() == sc_logic_1) {
        p_state.write(INIT);
    } else {
        p_state.write(n_state.read());
    }
}
}

```

e. Use a testbench to simulate your design in a SystemC simulation environment.

```

#ifndef DIVIDERTB_H
#define DIVIDERTB_H

#include <systemc.h>
#include "divider.h"

SC_MODULE(dividerTB){
    sc_signal<sc_logic> clk, rst, start;
    sc_signal< sc_lv<8> > inBus;
    sc_signal< sc_lv<8> > outBus;
    sc_signal<sc_logic> done;

```

```

divider_8bit* div;

SC_CTOR(dividerTB){
    div = new divider_8bit("Divider");
    div->clk(clk);
    div->rst(rst);
    div->start(start);
    div->inBus(inBus);
    div->outBus(outBus);
    div->done(done);

    SC_THREAD(inputing);
    SC_THREAD(reseting);
    SC_THREAD(clocking);

    SC_THREAD(displaying);
}
void inputing();
void resetting();
void clocking();
void displaying();
};

#endif

```

```

#include "dividerTB.h"

void dividerTB::clocking(){
    while(true){
        clk = SC_LOGIC_0;
        wait(5, SC_NS);
        clk = SC_LOGIC_1;
        wait(5, SC_NS);
    }
}

void dividerTB::reseting(){
    rst = SC_LOGIC_1;
    wait(12, SC_NS);
    rst = SC_LOGIC_0;
}

```

```

void dividerTB::inputing(){
    start = SC_LOGIC_0;
    inBus = "00000000";
    wait(20, SC_NS);

    start = SC_LOGIC_1;
    inBus = 37;
    wait(20, SC_NS);
    start = SC_LOGIC_0;
    inBus = 12;
    wait(10, SC_NS);

    wait(500, SC_NS);

    sc_stop();
}

void dividerTB::displaying(){
    while(true){
        wait(done.posedge_event());

        int quotient = outBus.read().to_uint();

        wait(clk.posedge_event());
        int remainder = outBus.read().to_uint();

        cout << "@" << sc_time_stamp() << " | Division Complete -> Quotient: "
             << quotient << ", Remainder: " << remainder << endl;
    }
}

```

```

#include <systemc.h>
#include "dividerTB.h"

int sc_main(int argc, char **argv){

    dividerTB TB1("testbench1");

    sc_trace_file* vcdfile;
    vcdfile = sc_create_vcd_trace_file("Divider_test");

```

```

vcdfile->set_time_unit(1, SC_NS);

sc_trace(vcdfile, TB1.clk, "clk");
sc_trace(vcdfile, TB1.rst, "rst");
sc_trace(vcdfile, TB1.start, "start");
sc_trace(vcdfile, TB1.inBus, "inBus");
sc_trace(vcdfile, TB1.outBus, "outBus");
sc_trace(vcdfile, TB1.done, "done");

sc_trace(vcdfile, TB1.div->DP->outA, "Reg_A");
sc_trace(vcdfile, TB1.div->DP->outQ, "Reg_Q");
sc_trace(vcdfile, TB1.div->CNTRL->p_state, "State");

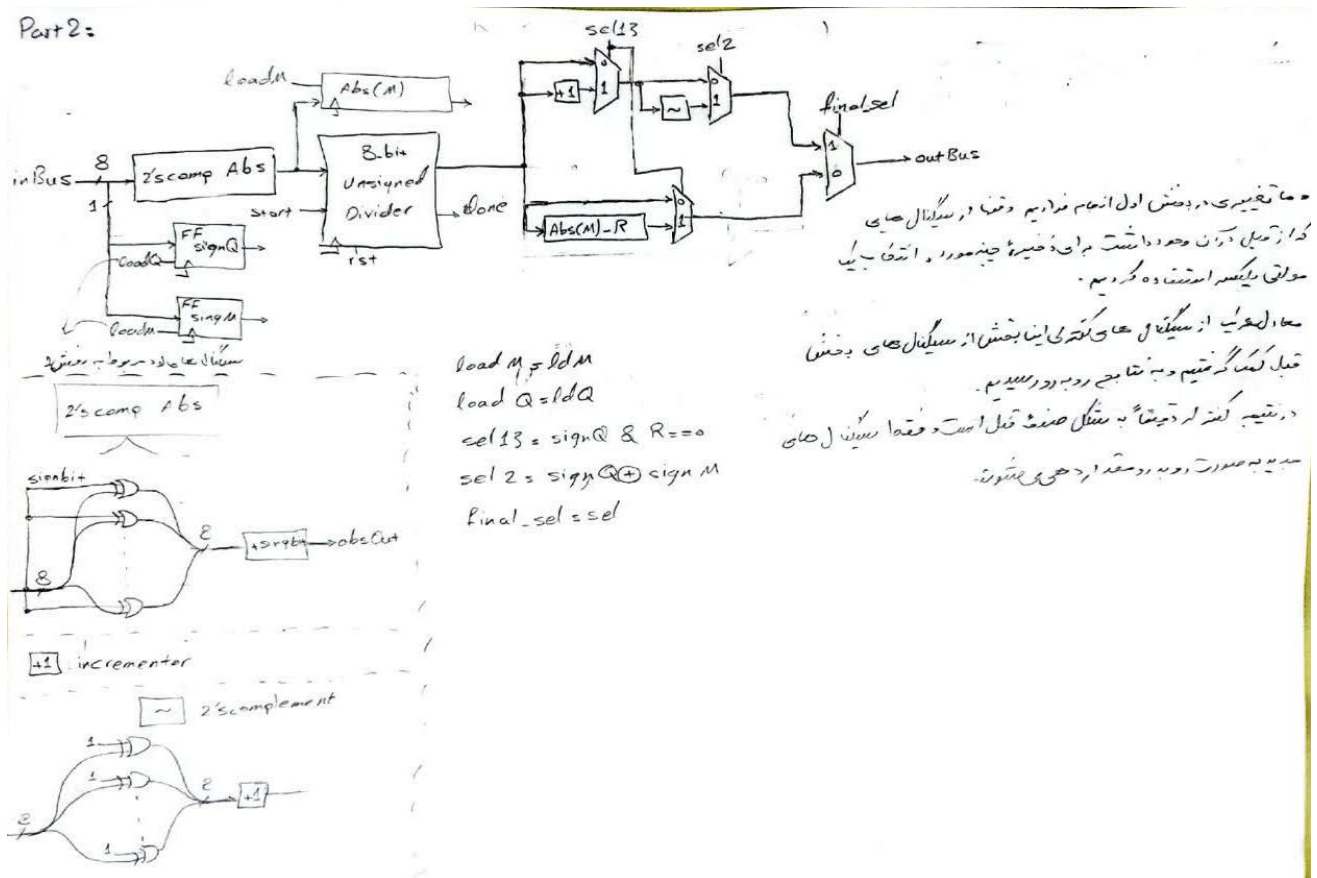
sc_start();

sc_close_vcd_trace_file(vcdfile);
return 0;
}

```



a. Using state machines, counters, shift-registers and other RTL components discussed in class show the design of the divider circuit.



b. Describe the sign divider in **SystemC**

```
#include <systemc.h>
#include "twosComplementAbs.h"
#include "part1/register.h"
#include "part1/divider.h"
#include "adder.h"
#include "subtractor_p2.h"
#include "negator.h"
#include "mux_p2.h"
#include "dff.h"

SC_MODULE(dividerSigned_8bit)
```

```

{
    sc_in<sc_logic> clk, rst, start;
    sc_in<sc_lv<8> > inBus;

    sc_out<sc_lv<8> > outBus;
    sc_out<sc_logic> done;

    sc_signal<sc_lv<8> > absOut;
    sc_signal<sc_lv<8> > divOut;

    sc_signal<sc_logic> signM;
    sc_signal<sc_logic> signQ;

    sc_signal<sc_logic> signM_in;
    sc_signal<sc_logic> signQ_in;

    sc_signal<sc_logic> R0;
    sc_signal<sc_logic> R0_reg;
    sc_signal<sc_logic> R0_en;

    sc_signal<sc_logic> sel13;
    sc_signal<sc_logic> sel2;

    sc_signal<sc_logic> loadM;
    sc_signal<sc_logic> loadQ;
    sc_signal<sc_logic> sel;
    sc_signal<sc_logic> dummyCarry;

    sc_signal<sc_lv<8> > divOutPlus1;
    sc_signal<sc_lv<8> > mux10Out;
    sc_signal<sc_lv<8> > mux10OutInvert;
    sc_signal<sc_lv<8> > finalQ;

    sc_signal<sc_lv<8> > TrueR;
    sc_signal<sc_lv<8> > finalR;
    sc_signal<sc_lv<8> > absM;

    sc_signal<sc_lv<8> > const_one;
    sc_signal<sc_lv<8> > const_zero;
    sc_signal<sc_logic> cin_zero;

    divider_8bit *div;
    twosComplementAbs *abs_unit;
    register_8bit *absMRegister;
    adder_8bit *q_incrementer;

```

```

subtractor_8bit_p2 *r_subtractor;

negator_8bit *q_negator;

mux_8bit_p2 *mux1;
mux_8bit_p2 *mux2;
mux_8bit_p2 *mux3;
mux_8bit_p2 *mux4;
mux_8bit_p2 *mux_final;

dff *signM_ff;
dff *signQ_ff;
dff *R0_ff;

void comb_logic()
{
    signM_in.write(inBus.read()[7]);
    signQ_in.write(inBus.read()[7]);

    sel2.write(signM.read() ^ signQ.read());

    if ((divOut.read().to_uint() == 0) & (sel.read() == SC_LOGIC_0))
        R0.write(SC_LOGIC_1);
    else
        R0.write(SC_LOGIC_0);

    sel13.write(signQ.read() & (R0.read() == SC_LOGIC_0));

    if (sel.read() == SC_LOGIC_0)
        R0_en.write(SC_LOGIC_1);
    else
        R0_en.write(SC_LOGIC_0);
}

void latch_results()
{
    // if (done.read() == SC_LOGIC_1)
    // {
    //     R0_reg.write(R0.read());
    // }
}

SC_CTOR(dividerSigned_8bit)
{
    cin_zero.write(SC_LOGIC_0);
}

```

```
const_one.write("00000001");
const_zero.write("00000000");

abs_unit = new twosComplementAbs("abs_unit");
abs_unit->in(inBus);
abs_unit->out(absOut);

absMRegister = new register_8bit("absM");
absMRegister->inData(absOut);
absMRegister->ld(loadM);
absMRegister->clk(clk);
absMRegister->outData(absM);

div = new divider_8bit("Divider");
div->clk(clk);
div->rst(rst);
div->start(start);
div->inBus(absOut);
div->outBus(divOut);
div->loadM(loadM);
div->loadQ(loadQ);
div->select(sel);
div->done(done);

signM_ff = new dff("signM_ff");
signM_ff->clk(clk);
signM_ff->rst(rst);
signM_ff->en(loadM);
signM_ff->d(signM_in);
signM_ff->q(signM);

signQ_ff = new dff("signQ_ff");
signQ_ff->clk(clk);
signQ_ff->rst(rst);
signQ_ff->en(loadQ);
signQ_ff->d(signQ_in);
signQ_ff->q(signQ);

R0_ff = new dff("R0_ff");
R0_ff->clk(clk);
R0_ff->rst(rst);
R0_ff->en(R0_en);
R0_ff->d(R0);
R0_ff->q(R0_reg);
```

```

q_incrementer = new adder_8bit("q_inc");
q_incrementer->a(divOut);
q_incrementer->b(const_one);
q_incrementer->cin(cin_zero);
q_incrementer->sum(divOutPlus1);
q_incrementer->cout(dummyCarry);

mux1 = new mux_8bit_p2("mux1");
mux1->a(divOut);
mux1->b(divOutPlus1);
mux1->sel(sel13);
mux1->y(mux10Out);

q_negator = new negator_8bit("q_neg");
q_negator->in(mux10Out);
q_negator->out(mux10OutInvert);

mux2 = new mux_8bit_p2("mux2");
mux2->a(mux10Out);
mux2->b(mux10OutInvert);
mux2->sel(sel2);
mux2->y(finalQ);

r_subtractor = new subtractor_8bit_p2("r_sub");
r_subtractor->a(absM);
r_subtractor->b(divOut);
r_subtractor->diff(TrueR);

mux3 = new mux_8bit_p2("mux3");
mux3->a(divOut);
mux3->b(TrueR);
mux3->sel(sel13);
mux3->y(finalR);

mux_final = new mux_8bit_p2("mux_final");
mux_final->a(finalR);
mux_final->b(finalQ);
mux_final->sel(sel);
mux_final->y(outBus);

SC_METHOD(comb_logic);
sensitive << inBus << signM << signQ << divOut << sel << R0;

SC_METHOD(latch_results);
sensitive << clk.pos();

```

```
    }  
};
```

c. Use a testbench to simulate your design in a SystemC simulation environment.

```
#ifndef DIVIDERSIGNEDTB_H  
#define DIVIDERSIGNEDTB_H  
  
#include <systemc.h>  
#include "dividerSigned.h"  
  
SC_MODULE(dividerSignedTB){  
  
    sc_signal<sc_logic> clk, rst, start;  
    sc_signal<sc_lv<8> > inBus;  
    sc_signal<sc_lv<8> > outBus;  
    sc_signal<sc_logic> done;  
  
    dividerSigned_8bit* div;  
  
    SC_CTOR(dividerSignedTB){  
  
        div = new dividerSigned_8bit("SignedDivider");  
  
        div->clk(clk);  
        div->rst(rst);  
        div->start(start);  
        div->inBus(inBus);  
        div->outBus(outBus);  
        div->done(done);  
  
        SC_THREAD(inputing);  
        SC_THREAD(reseting);  
        SC_THREAD(clocking);  
        SC_THREAD(displaying);  
    }  
  
    void inputing();  
    void resetting();  
    void clocking();  
    void displaying();  
};
```

```
#endif
```

```
#include "dividerSignedTB.h"

void dividerSignedTB::clocking(){
    while(true){
        clk = SC_LOGIC_0;
        wait(5, SC_NS);

        clk = SC_LOGIC_1;
        wait(5, SC_NS);
    }
}

void dividerSignedTB::reseting(){
    rst = SC_LOGIC_1;
    wait(12, SC_NS);

    rst = SC_LOGIC_0;
}

void dividerSignedTB::inputing(){

    start = SC_LOGIC_0;
    inBus = "00000000";

    wait(20, SC_NS);

    // test 1 : 27 / 5
    start = SC_LOGIC_1;
    inBus = 37;
    wait(10, SC_NS);

    inBus = 5;
    wait(10, SC_NS);

    start = SC_LOGIC_0;

    wait(400, SC_NS);

    // test 2 : -27 / 5
    // start = SC_LOGIC_1;
```

```

    // inBus = -27;
    // wait(10, SC_NS);

    // inBus = 5;
    // wait(10, SC_NS);

    // start = SC_LOGIC_0;

    // wait(200, SC_NS);

    // // test 3 : -40 / -6
    // start = SC_LOGIC_1;
    // inBus = -40;
    // wait(10, SC_NS);

    // inBus = -6;
    // wait(10, SC_NS);

    // start = SC_LOGIC_0;

    // wait(200, SC_NS);

    sc_stop();
}

void dividerSignedTB::displaying(){

    while(true){

        wait(done.posedge_event());

        int quotient = outBus.read().to_int();

        wait(clk.posedge_event());

        int remainder = outBus.read().to_int();

        cout << "@" << sc_time_stamp()
             << " | Division Complete -> Quotient: "
             << quotient
             << " , Remainder: "
             << remainder
             << endl;

    }
}

```

```
#include <systemc.h>
#include "dividerSignedTB.h"

int sc_main(int argc, char **argv){

    dividerSignedTB TB1("testbench1");

    sc_trace_file* vcdfile;
    vcdfile = sc_create_vcd_trace_file("DividerSigned_test");

    vcdfile->set_time_unit(1, SC_NS);

    sc_trace(vcdfile, TB1.clk, "clk");
    sc_trace(vcdfile, TB1.rst, "rst");
    sc_trace(vcdfile, TB1.start, "start");

    sc_trace(vcdfile, TB1.inBus, "inBus");
    sc_trace(vcdfile, TB1.outBus, "outBus");

    sc_trace(vcdfile, TB1.done, "done");

    sc_start();

    sc_close_vcd_trace_file(vcdfile);

    return 0;
}
```